

于某种给定的类型来说, name 的返回值因编译器而异并且不一定与在程序中使用的名字一致。对于 name 返回值的唯一要求是, 类型不同则返回的字符串必须有所区别。例如:

```
int arr[10];
Derived d;
Base *p = &d;

cout << typeid(42).name() << ", "
    << typeid(arr).name() << ", "
    << typeid(Sales_data).name() << ", "
    << typeid(std::string).name() << ", "
    << typeid(p).name() << ", "
    << typeid(*p).name() << endl;
```

在作者的计算机上运行该程序, 输出结果如下:

```
i, A10_i, 10Sales_data, Ss, P4Base, 7Derived
```



type_info 类在不同的编译器上有所区别。有的编译器提供了额外的成员函数以提供程序中所用类型的额外信息。读者应该仔细阅读你所用编译器的使用手册, 从而获取关于 type_info 的更多细节。

832

19.2.4 节练习

练习 19.9: 编写与本节最后一个程序类似的代码, 令其打印你的编译器为一些常见类型所起的名字。如果你得到的输出结果与本书类似, 尝试编写一个函数将这些字符串翻译成人们更容易读懂的形式。

练习 19.10: 已知存在如下的类继承体系, 其中每个类定义了一个默认公有的构造函数和一个虚析构函数。下面的语句将打印哪些类型名字?

```
class A { /* ... */ };
class B : public A { /* ... */ };
class C : public B { /* ... */ };

(a) A *pa = new C;
    cout << typeid(pa).name() << endl;
(b) C cobj;
    A& ra = cobj;
    cout << typeid(&ra).name() << endl;
(c) B *px = new B;
    A& ra = *px;
    cout << typeid(ra).name() << endl;
```

19.3 枚举类型

枚举类型 (enumeration) 使我们可以将一组整型常量组织在一起。和类一样, 每个枚举类型定义了一种新的类型。枚举属于字面值常量类型 (参见 7.5.6 节, 第 267 页)。

C++ 包含两种枚举: 限定作用域的和不限定作用域的。C++11 新标准引入了**限定作用域的枚举类型 (scoped enumeration)**。定义限定作用域的枚举类型的一般形式是: 首先是关键字 `enum class` (或者等价地使用 `enum struct`), 随后是枚举类型名字以及用花括

号括起来的以逗号分隔的**枚举成员**（enumerator）列表，最后是一个分号：

```
enum class open_modes {input, output, append};
```

我们定义了一个名为 `open_modes` 的枚举类型，它包含三个枚举成员：`input`、`output` 和 `append`。

定义**不限定作用域的枚举类型**（unscoped enumeration）时省略掉关键字 `class`（或 `struct`），枚举类型的名字是可选的：

```
enum color {red, yellow, green};           // 不限定作用域的枚举类型
// 未命名的、不限定作用域的枚举类型
enum {floatPrec = 6, doublePrec = 10, double_doublePrec = 10};
```

如果 `enum` 是未命名的，则我们只能在定义该 `enum` 时定义它的对象。和类的定义类似，我们需要在 `enum` 定义的右侧花括号和最后的分号之间提供逗号分隔的声明列表（参见 2.6.1 节，第 64 页）。

枚举成员

833

在限定作用域的枚举类型中，枚举成员的名字遵循常规的作用域准则，并且在枚举类型的作用域外是不可访问的。与之相反，在不限定作用域的枚举类型中，枚举成员的作用域与枚举类型本身的作用域相同：

```
enum color {red, yellow, green};           // 不限定作用域的枚举类型
enum stoplight {red, yellow, green};       // 错误：重复定义了枚举成员
enum class peppers {red, yellow, green};   // 正确：枚举成员被隐藏了
color eyes = green; // 正确：不限定作用域的枚举类型的枚举成员位于有效的作用域中
peppers p = green; // 错误：peppers 的枚举成员不在有效的作用域中
// color::green 在有效的作用域中，但是类型错误
color hair = color::red;                   // 正确：允许显式地访问枚举成员
peppers p2 = peppers::red;                 // 正确：使用 peppers 的 red
```

默认情况下，枚举值从 0 开始，依次加 1。不过我们也能为一个或几个枚举成员指定专门的值：

```
enum class intTypes {
    charTyp = 8, shortTyp = 16, intTyp = 16,
    longTyp = 32, long_longTyp = 64
};
```

由枚举成员 `intTyp` 和 `shortTyp` 可知，枚举值不一定唯一。如果我们没有显式地提供初始值，则当前枚举成员的值等于之前枚举成员的值加 1。

枚举成员是 `const`，因此在初始化枚举成员时提供的初始值必须是常量表达式（参见 2.4.4 节，第 58 页）。也就是说，每个枚举成员本身就是一条常量表达式，我们可以在任何需要常量表达式的地方使用枚举成员。例如，我们可以定义枚举类型的 `constexpr` 变量：

```
constexpr intTypes charbits = intTypes::charTyp;
```

类似的，我们也可以将一个 `enum` 作为 `switch` 语句的表达式，而将枚举值作为 `case` 标签（参见 5.3.2 节，第 160 页）。出于同样的原因，我们还能将枚举类型作为一个非类型模板形参使用（参见 16.1.1 节，第 580 页）；或者在类的定义中初始化枚举类型的静态数据成员（参见 7.6 节，第 270 页）。

和类一样，枚举也定义新的类型

只要 enum 有名字，我们就能定义并初始化该类型的成员。要想初始化 enum 对象或者为 enum 对象赋值，必须使用该类型的一个枚举成员或者该类型的另一个对象：

```
open_modes om = 2;           // 错误：2 不属于类型 open_modes
om = open_modes::input;      // 正确：input 是 open_modes 的一个枚举成员
```

834

一个不限定作用域的枚举类型的对象或枚举成员自动地转换成整型。因此，我们可以在任何需要整型值的地方使用它们：

```
int i = color::red; // 正确：不限定作用域的枚举类型的枚举成员隐式地转换成 int
int j = peppers::red; // 错误：限定作用域的枚举类型不会进行隐式转换
```

指定 enum 的大小

尽管每个 enum 都定义了唯一的类型，但实际上 enum 是由某种整数类型表示的。在

C++11

C++11 新标准中，我们可以在 enum 的名字后加上冒号以及我们想在该 enum 中使用的类型：

```
enum intValues : unsigned long long {
    charTyp = 255, shortTyp = 65535, intTyp = 65535,
    longTyp = 4294967295UL,
    long_longTyp = 18446744073709551615ULL
};
```

如果我们没有指定 enum 的潜在类型，则默认情况下限定作用域的 enum 成员类型是 int。对于不限定作用域的枚举类型来说，其枚举成员不存在默认类型，我们只知道成员的潜在类型足够大，肯定能够容纳枚举值。如果我们指定了枚举成员的潜在类型（包括对限定作用域的 enum 的隐式指定），则一旦某个枚举成员的值超出了该类型所能容纳的范围，将引发程序错误。

指定 enum 潜在类型的能力使得我们可以控制不同实现环境中使用的类型，我们将可以确保在一种实现环境中编译通过的程序所生成的代码与其他实现环境中生成的代码一致。

枚举类型的前置声明

C++11

在 C++11 新标准中，我们可以提前声明 enum。enum 的前置声明（无论隐式地还是显示地）必须指定其成员的大小：

```
// 不限定作用域的枚举类型 intValues 的前置声明
enum intValues : unsigned long long; // 不限定作用域的，必须指定成员类型
enum class open_modes; // 限定作用域的枚举类型可以使用默认成员类型 int
```

因为不限定作用域的 enum 未指定成员的默认大小，因此每个声明必须指定成员的大小。对于限定作用域的 enum 来说，我们可以不指定其成员的大小，这个值被隐式地定义成 int。

和其他声明语句一样，enum 的声明和定义必须匹配，这意味着在该 enum 的所有声明和定义中成员的大小必须一致。而且，我们不能在同一个上下文中先声明一个不限定作用域的 enum 名字，然后再声明一个同名的限定作用域的 enum：

```
// 错误：所有的声明和定义必须对该 enum 是限定作用域的还是不限定作用域的保持一致
enum class intValues;
enum intValues; // 错误：intValues 已经被声明成限定作用域的 enum
enum intValues : long; // 错误：intValues 已经被声明成 int
```

形参匹配与枚举类型

835

要想初始化一个 enum 对象，必须使用该 enum 类型的另一个对象或者它的一个枚举成员（参见 19.3 节，第 737 页）。因此，即使某个整型值恰好与枚举成员的值相等，它也不能作为函数的 enum 实参使用：

```
// 不限定作用域的枚举类型，潜在类型因机器而异
enum Tokens {INLINE = 128, VIRTUAL = 129};
void ff(Tokens);
void ff(int);
int main() {
    Tokens curTok = INLINE;
    ff(128);                      // 精确匹配 ff(int)
    ff(INLINE);                   // 精确匹配 ff(Tokens)
    ff(curTok);                   // 精确匹配 ff(Tokens)
    return 0;
}
```

尽管我们不能直接将整型值传给 enum 形参，但是可以将一个不限定作用域的枚举类型的对象或枚举成员传给整型形参。此时，enum 的值提升成 int 或更大的整型，实际提升的结果由枚举类型的潜在类型决定：

```
void newf(unsigned char);
void newf(int);
unsigned char uc = VIRTUAL;
newf(VIRTUAL);                  // 调用 newf(int)
newf(uc);                       // 调用 newf(unsigned char)
```

枚举类型 Tokens 只有两个枚举成员，其中较大的那个值是 129。该枚举类型可以用 unsigned char 来表示，因此很多编译器使用 unsigned char 作为 Tokens 的潜在类型。不管 Tokens 的潜在类型到底是什么，它的对象和枚举成员都提升成 int。尤其是，枚举成员永远不会提升成 unsigned char，即使枚举值可以用 unsigned char 存储也是如此。

19.4 类成员指针

成员指针（pointer to member）是指可以指向类的非静态成员的指针。一般情况下，指针指向一个对象，但是成员指针指示的是类的成员，而非类的对象。类的静态成员不属于任何对象，因此无须特殊的指向静态成员的指针，指向静态成员的指针与普通指针没有什么区别。

成员指针的类型囊括了类的类型以及成员的类型。当初初始化一个这样的指针时，我们令其指向类的某个成员，但是不指定该成员所属的对象；直到使用成员指针时，才提供成员所属的对象。

为了解释成员指针的原理，不妨使用 7.3.1 节（第 243 页）的 Screen 类：

836

```
class Screen {
public:
    typedef std::string::size_type pos;
    char get_cursor() const { return contents[cursor]; }
    char get() const;
```